



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de San Juan del Río



Tópicos de Ciberseguridad

**R005 PARCHAR UN EJECUTABLE
PRESENTA:**

Oscar Gabriel Rogelio Jiménez Hernández 22590332

Roberto Joshua García Solís 22590410

Ingeniería en Sistemas Computacionales

PROFESOR: Leonardo Valdez Arteaga

PERIODO [Agosto-Diciembre 2026]

1. Objetivo

Neutralizar la validación en línea del ejecutable `phackeame.exe` mediante ingeniería inversa, de modo que la aplicación conceda el acceso sin importar el usuario y la contraseña introducidos. El entregable es el **código fuente** del parche (un programa en Python), no el ejecutable modificado.

2. Herramientas utilizadas

- **Sistema operativo:** Linux (CachyOS, base Arch).
- **radare2** (o Cutter, su interfaz gráfica) para desensamblar, localizar y parchar la instrucción. El binario es PE de 32 bits.
- **Python 3** para automatizar el parche a partir del binario original.
- **Wine** para ejecutar el binario PE bajo Linux.
- Comparación binaria de archivos para validar el cambio.

3. Identificación del binario

Antes de analizar, se registran las propiedades del archivo para garantizar que se trabaja sobre la muestra correcta:

```
Archivo : phackeame.exe
Tipo    : PE32 executable (GUI) Intel 80386, MS Windows, 8 secciones
Tamaño  : 2,128,915 bytes (0x207C13)
SHA256  : 160c6af0d3e86ec0f1eef57f00c84ae6b5969a8a46c7caf1506d12516047a00c
```

La aplicación fue compilada con Lazarus/Free Pascal, coherente con la sugerencia de la actividad (Lazarus + Python para el servicio de validación).

4. Análisis de la validación en línea

La aplicación presenta un formulario de inicio de sesión (Usuario / Contraseña). Al examinar las cadenas del binario se identifica que **envía las credenciales a un servidor por HTTP** y decide el acceso según la respuesta:

- Endpoint de validación: `http://45.76.173.114:8080/login`
- Cuerpo de la petición en formato JSON con los campos USER y PASS (se observa el fragmento `"PASS":` en las cadenas).
- Destino tras el acceso: `http://nvubank/home`

El programa **compara el código de respuesta HTTP contra 200 (HTTP 200 OK)**. Este es el punto exacto donde se decide conceder o negar el acceso, y por lo tanto el objetivo del parche.

```
~/Descargas/exe
) rabin2 -z phackeame.exe | grep -Ei 'login|45\.76|PASS|USER|nvubank'
161 0x00163482 0x00564e82 96 97 .data ascii
1010 0x0017eb25 0x00581325 8 9 .rdata ascii \atxtpass
1011 0x0017eb33 0x00581333 8 9 .rdata ascii \aLbLpass
1015 0x0017eb94 0x00581394 10 11 .rdata ascii ", "PASS": "
1016 0x0017ebbc 0x005813bc 31 32 .rdata ascii http://45.76.173.114:8080/login
1025 0x0017ed40 0x00581540 19 20 .rdata ascii http://nvubank/home
1028 0x0017edc4 0x005815c4 10 11 .rdata ascii User-Agent
1431 0x001848d8 0x005870d8 10 11 .rdata ascii \tEBusError
1712 0x00186df9 0x005895f9 13 14 .rdata ascii \tEBusErrorppX
1904 0x00188a2c 0x0058b22c 18 19 .rdata ascii sysconst.sbusererror
2291 0x0018dfa0 0x005907a0 18 19 .rdata ascii TaskDiaLogIndirect
3747 0x001a05f0 0x005a2df0 10 11 .rdata ascii user32.dll
3797 0x001a0d60 0x005a3560 6 7 .rdata ascii USER32
4543 0x001af982 0x005b2182 26 27 .rdata ascii emNone\nemPassword\bStdCtrls
4651 0x001b0876 0x005b3076 14 15 .rdata ascii 4\FPasswordChar
```

5. Localización de la instrucción en el depurador

Cargando el binario en radare2 y rastreando las cadenas hasta la rutina de validación, se llega a la dirección 0042A266. El bloque relevante es:

Dirección	Bytes	Instrucción (ensamblador)
0042A25F	81 7D E4 C8 00 00 00	cmp dword ptr [ebp-1C], 0C8h
0042A266	75 1C	jne 0042A284 ← salto a neutralizar
0042A268	8B 45 F8 ...	mov eax, [ebp-8] ... mov dl, 1 (concede acceso)

cmp dword ptr [ebp-1C], 0C8h compara el valor obtenido contra 0xC8 (= 200 decimal). Si son iguales, jne *no* salta y la ejecución continúa hacia el bloque que ejecuta mov dl, 1, el cual habilita el acceso. Si *no* son iguales, jne salta a 0042A284, omitiendo por completo ese bloque y negando el acceso.

Maapeo de dirección virtual a offset físico (necesario para el parche en Python):

ImageBase = 0x00400000 sección .text: RVA 0x1000, RawPtr 0x400

VA 0x0042A266 → RVA 0x2A266 → offset de archivo = 0x2A266 - 0x1000 + 0x400 = 0x29666

```
~/Descargas/exe
) ls
.rwxr-xr-x 2.1M chino 10 sep 20:28 phackeame.exe

~/Descargas/exe
) r2 -q -c 's 0x0042A266; pd 3' phackeame.exe
WARN: Relocs has not been applied. Please use '-e bin.relocs.apply=true' or '-e bin.cache=true' next time
< 0x0042a266 751c jne 0x42a284
0x0042a268 8b45f8 mov eax, dword [ebp - 8]
0x0042a26b 8b80b0040000 mov eax, dword [eax + 0x4b0]

~/Descargas/exe
)
```

6. Aplicación del parche

La idea del parche es **eliminar el salto condicional**: si el salto nunca ocurre, la ejecución siempre cae en el bloque que concede el acceso. Para ello se reemplazan los dos bytes de la instrucción `jne` (que ocupa `75 1C`) por dos instrucciones `nop` (`90 90`), que no hacen nada:

	Bytes	Significado
Original	75 1C	<code>jne 0042A284</code>
Parchado	90 90	<code>nop / nop</code>

En el depurador se edita la instrucción en `0042A266` escribiendo los bytes `90 90`. El resultado en desensamblado queda como dos `nop` consecutivos, y la comparación previa deja de tener efecto sobre el flujo.

```
~/Descargas/exe
) cp phackeame.exe phackeame_parche.exe

~/Descargas/exe
) r2 -w -q -c 's 0x0042A266; wx 9090; pd 2' phackeame_parche.exe
WARN: Relocs has not been applied. Please use '-e bin.relocs.apply=true' or '-e bin.cache=true' next time
0x0042a266 90      nop
0x0042a267 90      nop
```

7. Automatización con Python

La actividad pide un programa que lea el ejecutable original y aplique el cambio. El script `parche_phackeame.py` localiza el offset `0x29666`, verifica que ahí estén exactamente los bytes esperados `75 1C` y los sustituye por `90 90`, generando un archivo nuevo sin modificar el original. Lógica principal:

- Verifica que el archivo original exista y que el offset esté dentro del archivo.
- Comprueba los bytes en `0x29666`: si ya son `90 90` avisa que ya estaba parchado; si no son `75 1C` se detiene (no es el binario correcto).
- Crea un respaldo `.bak` del original antes de cualquier cambio.
- Escribe `phackeame_parche.exe` con los dos bytes sustituidos.

El código fuente completo se entrega en el archivo `parche_phackeame.py` que acompaña a este reporte.

```
~/Descargas/exe
) python parche_phackeame.py
[OK] Parche aplicado correctamente.
Offset      : 0x29666
Cambio      : 75 1c (JNE) -> 90 90 (NOP NOP)
Respaldo    : /home/chino/Descargas/exe/phackeame.exe.bak
Archivo nuevo : /home/chino/Descargas/exe/phackeame_parche.exe
```

8. Comparación del original contra el parchado

Aplicando una comparación binaria (diff) entre `phackeame.exe` y `phackeame_parche.exe` se confirma que **solo cambian dos bytes**, en el offset esperado, y que el tamaño del archivo no varía:

Tamaño idéntico : sí

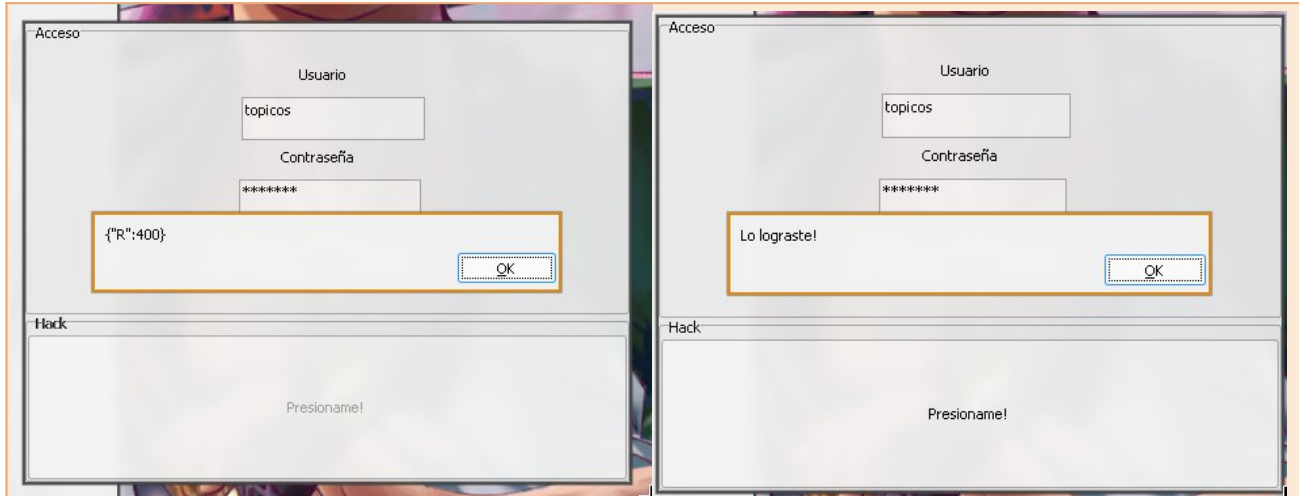
Bytes distintos : `0x29666` (`75 → 90`) y `0x29667` (`1C → 90`)

Esto demuestra que el parche es mínimo y no altera ninguna otra parte del ejecutable.

```
~/Descargas/exe
) cmp -l phackeame.exe phackeame_parche.exe
169575 165 220
169576 34 220
```

9. Resultados

1. Al ejecutar phackeame_parche.exe e introducir cualquier usuario y contraseña, la aplicación concede el acceso.
2. El salto condicional que dependía de la respuesta del servidor quedó neutralizado, por lo que la validación en línea ya no determina el resultado.
3. Se cumple el criterio de la actividad: “el ejecutable debe funcionar no importa qué usuario y contraseña use”.



10. Conclusión y reflexión ética

El ejercicio muestra cómo una validación basada únicamente en una comprobación local del código de respuesta puede ser evadida modificando un solo salto condicional en el binario. La lección de seguridad es clara: la lógica de autorización crítica no debe residir del lado del cliente, ya que un ejecutable en manos del usuario siempre puede ser inspeccionado y modificado. Mecanismos como validación del lado del servidor, firma de código y verificación de integridad reducen este riesgo.